

Detection Lab — Platform Integration Requirements

Detection Lab is a Flyreel-internal tool for building, evaluating, and refining VLM-based visual detection prompts (e.g. roof corrosion, missing siding, hazard identification). It supports the full lifecycle of a detection: building labeled image datasets, annotating them through a structured QA pipeline, drafting and iterating prompts against those datasets, comparing prompt versions, and running held-out evaluations before promoting a detection to production.

The current POC runs locally on SQLite with file-system storage and no authentication. This document captures the requirements to harden the POC into a hosted, multi-user, role-gated platform tool integrated alongside the existing Flyreel dashboard. The work is split into three milestones, each delivering a coherent MVP:

- **Milestone 1 — Annotation MVP:** General platform integration + Datasets, Annotation, and QA tabs. Delivers a production-grade labeling pipeline that produces auditable, finalized datasets. - **Milestone 2 — Prompt Building MVP:** Detection Setup, Build & Run Datasets, HiL Review, Detections & Logs, and Admin tabs. Delivers the core prompt authoring loop on top of the finalized datasets from M1. - **Milestone 3 — Prompt Enhancement & Evaluation MVP:** Prompt Feedback, Prompt Compare, and Held-Out Eval tabs. Delivers automated improvement suggestions, side-by-side prompt comparison, and the final regression gate before deploying a detection.

The example screenshots referenced in each section live in the project root (`screenshots/` and `Screenshot 2026-06-XX at ...png`).

Engineers should treat the existing POC as a working reference implementation — the goal of this rebuild is platform integration, not a re-design. UI behaviors described below should be reproduced unless explicitly flagged for change.

Cross-Cutting Concepts (applies to all tabs)

Before the per-tab requirements, three concepts apply globally and should be reflected throughout the implementation:

Image ID Uniqueness

Image IDs must be globally unique across the platform, with one exception: a **parent dataset and any datasets linked to it as children** (e.g. duplicate-annotation copies created for QA discrepancy review) share the same image IDs by design, so that label disagreements can be computed image-for-image. Uploading an image whose ID already exists outside of a parent-child link should be rejected at the API layer with a clear error.

Annotator User Role (NEW)

A new platform user role — **Annotator** — must be created. Annotators have access **only to the Annotation tab**. They cannot see Datasets, Detection Setup, runs, prompts, logs, or any other surface in Detection Lab. The existing **Super Admin** group retains full access to every tab and every action.

Annotator Identity & Impersonation

- The annotator selector dropdown (visible in the Annotation tab and in any "assign to annotator" modal) is populated from the membership of the **Annotator** user group in the platform — not from a hand-maintained list. - When a user logged in as **Annotator** opens the Annotation tab, they only ever see their own dashboard; the annotator selector is hidden or locked to their own identity. - When a user logged in as **Super Admin** opens the Annotation tab, they can use the annotator selector to **impersonate** any annotator and see exactly what that annotator's dashboard looks like (assigned datasets, flags, performance). Impersonation is read-through (Super Admin actions taken under impersonation are logged as Super Admin, not as the impersonated annotator).

Section 1: General App Requirements

This section covers cross-cutting platform-integration requirements that are independent of any single tab. It expands the requirements table the user already drafted (authentication, RBAC, data persistence, hosting, storage, security). All items below are **Milestone 1**.

#	Requirement	Details	User Story	Acceptance Criteria	Milestone
1.1	Enforced Authentication	All users must authenticate via the existing Flyreel SSO/auth provider before accessing any part of Detection Lab. No anonymous access.	As a user, I must log in before accessing the tool so that access is controlled.	Unauthenticated users are redirected to login; all API requests without valid auth return 401.	M1
1.2	Session Management	Secure session handling with expiration and logout invalidation, matching the rest of the Flyreel dashboard.	As a user, I want my session to be secure and expire appropriately.	Sessions expire after the configured platform timeout; logout invalidates the session token immediately.	M1
2.1	User Model	System maintains a user record (id, email, role, group membership, timestamps).	As a system, I need to track users to enforce permissions.	User records are created on first login and persisted in DB.	M1
3.1	Role-Based Access Control	System must support Super Admin , Prompt Engineer , Data Analyst , and Annotator roles. See role matrix below.	As an admin, I want to control who can access and modify data.	Role assignments persist; unauthorized actions return 403; UI reflects role.	M1
3.2	API-Level Enforcement	All API endpoints enforce role permissions, not just the UI.	As a system, I must prevent unauthorized data changes.	Attempting restricted actions returns 403 consistently across endpoints.	M1
3.3	UI Permission Enforcement	Frontend hides/disables actions the user cannot take. Annotators see only the Annotation tab in the nav; non-Super-Admin roles never see Admin tab controls.	As a user, I only see actions I am allowed to take.	UI elements (buttons/actions) are hidden or disabled based on role.	M1
3.4	Annotator Group Drives Selector	Annotator dropdowns (Annotation tab header, "Assign to Annotators" modal in Saved Datasets) are populated from membership in the platform's Annotator user group.	As an admin, I want adding/removing annotators in the platform to automatically reflect in Detection Lab.	New members of the Annotator group appear in dropdowns within one session refresh; removed members disappear.	M1
3.5	Super Admin Impersonation	Super Admin users can view the Annotation tab as any annotator from the annotator selector, seeing that annotator's dashboard exactly as the annotator would.	As a Super Admin, I want to QA an annotator's experience without sharing credentials.	Selecting an annotator while logged in as Super Admin shows that annotator's assigned datasets, flags, and performance metrics; any write actions taken while impersonating are logged with the Super Admin's identity.	M1
4.1	Shared Workspace Model	All detections, prompts, datasets, runs, flags, and QA records are globally visible to authorized users — no per-user silos.	As a user, I want to see the same data as my team.	Multiple users see identical data across sessions/devices.	M1
4.2	Cross-Session Persistence	Data persists across logins and devices.	As a user, I want my work to persist and be accessible later.	Data created by User A is visible to User B after login.	M1

#	Requirement	Details	User Story	Acceptance Criteria	Milestone
4.3	Concurrency Handling	System must handle concurrent edits without silent overwrites. Define optimistic locking or last-write-wins per entity (recommended: optimistic on Detection, PromptVersion, Dataset; last-write-wins on per-image labels with a "last edited by" stamp).	As a user, I want to avoid losing work due to concurrent edits.	Simultaneous updates do not corrupt records; conflicting writes surface a clear error to the second writer.	M1
4.4	Globally Unique Image IDs	Image IDs are unique platform-wide. The only exception: a parent dataset and any datasets linked to it as children (duplicate-annotation copies for QA) share the same image IDs.	As a system, I need stable image identity so labels and predictions can be joined across datasets and runs.	Uploading an image with a duplicate ID outside a parent-child link returns a clear validation error; parent-child linked datasets accept the shared IDs without error.	M1
5.1	Migration to Production DB	Replace SQLite with PostgreSQL (same Postgres cluster as the rest of the platform).	As a system, I need a scalable database for multiple users.	App runs against external Postgres; no local DB dependency remains.	M1
5.2	Data Migration	Existing SQLite data must be migrated to Postgres. *Confirm with Delaney at migration time which datasets/prompts/run logs to bring forward — not all need to migrate.*	As a user, I do not lose existing in-scope work during upgrade.	All in-scope datasets, prompts, and runs are preserved post-migration.	M1
6.1	Cloud Object Storage	Replace local file storage with GCS (or S3, matching platform convention).	As a system, I need durable storage for dataset images.	Uploaded files persist across deployments and instances.	M1
6.2	Secure File Access	Images are not publicly exposed; access uses signed URLs. *@Yong Deng — confirm whether this is still required when datasets are built from signed Metabase URLs.*	As a user, I want secure access to uploaded data.	Direct public access is blocked; access requires signed URLs.	M1
7.1	Hosted Deployment	Detection Lab runs as a hosted service alongside the Flyreel dashboard, not locally.	As a user, I access the tool without running it locally.	App accessible via a Flyreel URL; not dependent on localhost runtime.	M1
7.2	Stateless Services	Application instances must be stateless so multiple instances can run behind a load balancer.	As a system, I scale horizontally.	Multiple instances run without data inconsistency.	M1
8.1	Authenticated APIs	All API endpoints require authentication.	As a system, I must secure all endpoints.	Unauthenticated requests return 401.	M1
8.2	Rate Limiting	Rate limiting on write-heavy endpoints, but the Gemini inference endpoint must support runs of up to 600 images in sequence without throttling.	As a system, I prevent abuse while supporting our largest legitimate runs.	Excess requests are throttled; a single run of 600 images completes without rate-limit errors.	M1

#	Requirement	Details	User Story	Acceptance Criteria	Milestone
8.3	Formal Security Review	System passes internal security review before production. *@Yong Deng — define what this review entails based on the technical spike.*	As a company, we ensure the tool meets security standards.	Review completed and signed off.	M1
8.4	Secret Management	All secrets (Gemini API key, DB credentials, signed-URL secrets) managed via the platform's existing env/config system.	As a system, I protect sensitive credentials.	No secrets present in codebase or logs; rotated via standard platform tooling.	M1
9.1	Audit Logging	All write actions (dataset creation, label changes, flag resolution, prompt version creation, QA status transitions, impersonation sessions) are logged with actor, timestamp, and entity.	As an admin, I want to trace who changed what and when.	Audit log queryable from backend; surfaced in QA "Activity Log" tab.	M1

Role Matrix

Action	Super Admin	Prompt Engineer	Data Analyst	Annotator
View all tabs	✓	✓ (except Admin)	✓ (except Admin)	✗ (Annotation tab only)
Edit Detection Setup, Prompts	✓	✓	✗	✗
Edit Datasets (upload, append, link)	✓	✓	✓	✗
Annotate (label images, flag items)	✓	✓	✓	✓
Resolve QA flags, transition QA status	✓	✓	✗	✗
Run inference, run held-out eval	✓	✓	✗	✗
Delete datasets / prompts / runs / prompt versions	✓	✗	✗	✗
Edit Admin templates & limits	✓	✗	✗	✗
Manage user permissions	✓	✗	✗	✗
Impersonate annotators	✓	✗	✗	✗

MILESTONE 1 — Annotation MVP

Milestone 1 delivers a production-grade labeling pipeline: annotators can be assigned datasets, label them image-by-image, flag uncertain items, and QA managers can sample, resolve flags, finalize datasets, and track annotator performance — all under platform auth and RBAC. After M1,

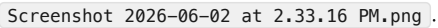
Detection Lab is a usable annotation tool even before any prompt-building features are migrated.

The three tabs in scope for M1 are **Saved Datasets**, **Annotation**, and **QA**. The prompt-related tabs are intentionally deferred to M2 so M1 can ship without depending on inference infrastructure.

Section 2: Saved Datasets Tab (M1)

Purpose. Saved Datasets is the centralized hub for managing all image collections in Detection Lab. It is where datasets are inspected, edited, assigned to annotators, linked together for duplicate-annotation QA, exported, and deleted. Every other tab that needs a dataset reads it from here.

Workflow. 1. A super admin user opens the Saved Datasets tab and either: a. Builds a new dataset by naming it, optionally assigning it to a detection, selecting a split type (MASTER, TRAIN, TEST, EVALUATE, CUSTOM), and uploading images (files / CSV); or b. Opens an existing dataset to inspect, edit metadata, or assign it to annotators and create linked child datasets. 2. The user assigns the dataset to one or more annotators from the platform Annotator group. Assignment creates a copy of the dataset (with optional label/attribute reset) for each assigned user and routes the assigned dataset to that annotator. 3. When QA later approves the separate child datasets after performing QA and label/attribute discrepancy reconciliation, the child datasets are merged back into the parent dataset, shifting its status to finalized and making it available as input to runs (M2) and held-out evals (M3). The child datasets are then archived.

Reference screenshots:  ,  .

Requirement	Details	User Story	Notes	Designs
2.1 – Create New Dataset	"Create Dataset" action opens a build flow supporting two upload methods: 1. Drag-and-drop image files. 2. CSV upload with columns <code>imageId</code> , <code>imageUrl</code> , <code>groundTruthLabel</code> , <code>attributes</code> . After upload, the user names the dataset, picks a split type, optionally assigns it to a detection, and saves.	As a system admin, I want to spin up a new dataset from a folder of images or a CSV created from a Metabase export of image URLs.	Image IDs must be globally unique on upload. Reject duplicates with a clear error pointing at the conflicting dataset. When CSVs are uploaded, they must have the following columns: <code>imageId</code> , <code>imageUrl</code> , <code>groundTruthLabel</code> , <code>attributes</code> . Include ability to download sample CSV: <code>dataset-manifest-example.csv</code> . Image ID and URL must be populated; <code>groundTruthLabel</code> and <code>attributes</code> can be left blank. Support the following dataset split types: MASTER, TRAIN, TEST, EVALUATE, CUSTOM. Button label: "Create Dataset". Pressing "Upload Dataset" once files are attached creates the dataset and closes the creation form.	Create Dataset button, Image files upload, CSV manifest upload

Requirement	Details	User Story	Notes	Designs
2.2 – Dataset Table	Top-level list of all datasets. Columns: Name (30%), Detection (20%), Split Type as badge (10%), Size as item count (8%, centered), Status as badge (16%), Updated as date (16%). Sortable on Name, Size, Updated. Paginated with configurable page size (5, 10, 25, 50).	As a user, I want to find and triage datasets quickly.	Parent-child hierarchy: parent datasets show a chevron toggle to expand/collapse their child datasets, plus a link icon. Children render indented with a "└" prefix and muted text. Full-width tab filter above the table: All (excludes archived), Drafts (draft status), Processing (assigned, in annotation, needs revision), In Review (submitted, in QA, approved), Finalized (finalized status), Archived (archived status). Clicking a row selects the dataset and opens the Details Panel below.	<i>Dataset table with filter tabs, parent-child hierarchy</i>
2.3 – Status Badges	Use a consistent color-coded status set across the app: Draft (gray), Assigned (blue), In Annotation (blue), Submitted (amber), In QA (amber), Needs Revision (red), Approved (green), Finalized (green), Archived (slate).	As a user, I want to read status at a glance anywhere in the app.	The same badge component and color scheme must be used consistently across Saved Datasets, Annotation, and QA tabs. Badge classes: <code>app-badge-muted</code> (gray), <code>app-badge-info</code> (blue), <code>app-badge-warning</code> (amber), <code>app-badge-danger</code> (red), <code>app-badge-success</code> (green).	
2.4 – Expandable Row → Details Panel	Clicking a row in the Dataset Table opens a Details Panel below the table showing: dataset name as title, an Actions menu (top-right), metadata (detection, split type, status, labeled count/percentage), image attributes section, and a scrollable items table with columns: Preview (thumbnail), Image ID (sortable), Image Description, Ground Truth Label (sortable, centered), Attributes (segment tags), Action (remove button, edit mode only).	As a user, I want to inspect a dataset's contents without leaving this page.	Thumbnails lazy-load. Clicking a thumbnail opens the shared Image Preview Modal. Items table sorts by Image ID or Ground Truth Label (ascending/descending toggle). Maximum height of 360px with scroll. Empty state displays "No images in this dataset." when no items exist.	<i>Details Panel with items table</i>
2.5 – Actions Menu	In the upper-right corner of the Details Panel header, an "Actions" dropdown ( icon) provides the following options: Auto-Split MASTER (only when split_type = MASTER, disabled if dataset empty or in progress), Edit Details / Save Changes (toggles between view and edit mode), Cancel Edit (only when editing, reloads items from server), Export CSV , Export JSON (both disabled if dataset has 0 items), Assign Annotators (disabled if dataset has 0 items), separator, Delete Dataset (red text).	As a user, I want a single access point for all dataset-level operations.	Menu items disable contextually (e.g., export and assign disabled on empty datasets, auto-split only on MASTER). Save Changes replaces Edit Details while editing. Cancel reverts all unsaved changes by reloading from the server.	<i>Actions dropdown menu</i>

Requirement	Details	User Story	Notes	Designs
2.6 – Edit Dataset Details	Selecting "Edit Details" from the Actions menu enters edit mode. Editable fields: Dataset Name (text input). Detection (dropdown: "Unassigned" + all detections). Split Type (dropdown: MASTER, TRAIN, TEST, EVALUATE, CUSTOM). Image Attributes (add/remove attributes from the detection's available set — see 2.7). Per-item fields also become editable: image_id (text input, sanitized), image_description (text input), ground_truth_label (select: UNSET / DETECTED / NOT_DETECTED), segment_tags (SegmentTagsEditor), plus a Remove button per item.	As a user, I want to correct metadata, relabel images, and manage attributes after initial creation.	Saving validates: no blank image IDs, no duplicate image IDs within the dataset. Sends two API calls: one PUT for metadata (name, detection_id, split_type) and one PUT with <code>action: "bulk_update_items"</code> for item-level changes. Changing detection on a non-empty dataset should warn that segment taxonomies may not match.	<i>Edit mode with per-item fields</i>
2.7 – Image Attributes Editor	When in edit mode with a detection assigned, an "Image Attributes" section appears between the metadata form and the items table. Shows the detection's full attribute taxonomy as removable tags (x to delete). Includes a text input to add new attributes and "Add" / "Save Attributes" buttons. In view mode, the same section displays the attribute set as read-only pills.	As a user, I want to manage which attributes are available to tag images with, directly from the dataset details view.	Adding/removing attributes here edits the detection's <code>segment_taxonomy</code> (persisted via <code>PUT /api/detections</code>). Changes are immediately reflected in the per-image SegmentTagsEditor options. "Save Attributes" is a separate save action from "Save Changes" — it updates the detection, not the dataset. Enter key in the input field adds the attribute. Duplicate attributes (case-insensitive) are rejected silently. Count displayed as "{N} total".	<i>Attribute editor with tag pills and add input</i>
2.8 – Auto-Split MASTER	When a MASTER dataset has all items labeled, "Auto-Split MASTER" in the Actions menu creates three child datasets: TRAIN (50%), TEST (20%), and EVALUATE (30%). Split is stratified by ground truth label and balanced by segment tags where available.	As a user, I want to create balanced train/test/eval splits from a fully labeled MASTER without manual partitioning.	Prerequisites: dataset must be split_type = MASTER, all items must have ground_truth_label set, at least 1 item must exist, no blank/duplicate image IDs. Derived names: <code>"{baseName} (TRAIN)"</code>, <code>"{baseName} (TEST)"</code>, <code>"{baseName} (EVALUATE)"</code>. Confirmation dialog warns if matching names already exist. Algorithm shuffles items, separates by label, and allocates to splits with attribute-balancing penalties.	

Requirement	Details	User Story	Notes	Designs
2.9 – Export Dataset	Export options available from the Actions menu. Export CSV: downloads one row per image with columns <code>imageId</code> , <code>imageUrl</code> , <code>groundTruthLabel</code> , <code>attributes</code> (JSON array format). Export JSON: downloads a structured object containing the full dataset record and an items array with <code>image_id</code> , <code>image_uri</code> , <code>image_description</code> , <code>ground_truth_label</code> , <code>segment_tags</code> .	As a Data Analyst, I want to hand off finalized datasets for downstream use in a format that can be re-imported or analyzed externally.	Available to all roles with view access. Disabled when the dataset has 0 items. CSV format matches the import template (<code>dataset-manifest-example.csv</code>) so exported CSVs can be re-imported directly. Filename: " <code>{sanitized_dataset_name}.csv</code> " or " <code>.json</code> ". Attributes value "Baseline" is filtered out of CSV export.	
2.10 – Assign to Annotators Modal	"Assign Annotators" opens a modal titled "Assign Annotators" with description "Create annotation copies of '{name}' for selected annotators." Lists all members of the platform Annotator user group as a checklist. Already-assigned annotators are shown as disabled (checked, non-removable). Two checkboxes: "Reset ground truth labels" and "Reset segment attributes". On assign, a child dataset is created per selected annotator, sharing image IDs with the parent.	As a Super Admin, I want to send the same images to multiple annotators for independent labeling.	Annotator list must come from the live platform Annotator group — no hand-maintained list. Assigning creates child datasets with <code>qa_status = "assigned"</code> visible in the annotator's Annotation tab. Submit button label: "Create {N} Assignment(s)" (dynamically shows count). Disabled until at least one annotator is selected. Child datasets are linked to the parent via <code>linked_dataset_id</code> .	<i>Assign Annotators modal with checklist</i>
2.11 – Delete Dataset	Delete button in the Actions menu (red text, positioned after a visual separator). Opens a confirmation dialog: title "Delete Dataset", message "Delete dataset '{name}'? This cannot be undone.", confirm button "Delete Dataset" with danger tone. Hard deletes the dataset and all its items.	As a Super Admin, I want to clean up draft or obsolete datasets.	Post-delete: clears selection, reloads dataset list, triggers app-wide refresh. Future requirement: should be restricted to Super Admin role only and disabled if the dataset has associated runs (M2+). Deleting a parent dataset that has linked children should refuse and force user to unlink first.	
2.12 – Image Preview Modal	Shared modal across all tabs. Full-size image display with navigation (prev/next buttons and arrow key support). Side panel displays: Image ID, Image Description, Ground Truth Label (with edit buttons in edit mode: DETECTED / NOT_DETECTED / UNSET), Attributes (SegmentTagsEditor in edit mode, SegmentTagList in view mode), Flag section (flag button or existing flag display with resolve option), and Resolved Flag History (original reason, resolution action, resolution note, date).	As a user, I want to inspect an image at full size with all its metadata without leaving the table view.	Same component is reused in Annotation, QA, and HiL Review tabs. Navigation: left/right arrow keys, up/down arrow keys. Escape closes the modal. Index counter displayed as " <code>{current}/{total}</code> ". In edit mode, all per-item fields are editable directly from the modal.	<i>Image Preview Modal with details side panel</i>

Requirement	Details	User Story	Notes	Designs
2.13 – Review Flags	From the Image Preview Modal or items table, users can "Flag for Secondary Review" — opening a modal with a reason textarea and "Submit Flag" button (disabled until reason provided). Flagged items display an indicator. Flags can be resolved via a "Resolve Flag" modal with a resolution action dropdown (Label Confirmed, Label Corrected, Attributes Corrected, Image Removed, Needs Discussion) and an optional note.	As a user, I want to surface uncertain or problematic images for review without blocking the entire dataset workflow.	Flag and Resolve modals are the same components used in the QA and Annotation tabs. Resolved flags display their full history (original reason, resolution action, note, date) in both the items table and the Image Preview Modal.	
2.14 – Empty / Loading States	Empty state when no datasets exist: informational copy + "Create Dataset" CTA. Skeleton rows or loading indicator while data is being fetched. Empty items state: "No images in this dataset." message in the details panel.	As a user, I want clear feedback when the table is empty or loading so I know the system is working.		

Section 3: Annotation Tab (M1)

Purpose. The Annotation tab is the labeling workspace. It is the **only** tab visible to users in the Annotator role. Annotators see datasets that have been assigned to them, work through them image-by-image to assign ground-truth labels and segment tags, leave notes, and flag images they cannot confidently label. They can also review their own flags and personal performance metrics. Super Admins can use this tab in impersonation mode to QA any annotator's experience.

Workflow. 1. The annotator logs in and lands on the Annotation tab → My Datasets sub-tab. They see four summary cards (Assigned / Needs Attention / In Progress / Complete) and a filterable table of their assigned datasets. 2. They click a dataset to enter the image-by-image annotation view. The viewer is a two-column layout: the image on the left with zoom/pan and keyboard navigation, the review panel on the right with the label buttons, segment-tag toggles, notes textarea, and a "Flag for Secondary Review" action. 3. They progress through images using arrow keys or the prev/next buttons. Notes auto-save on blur or navigation. The progress counter updates live. 4. When done, they hit Submit; the dataset moves to `submitted` status and is no longer editable to them. Submitted datasets become QA's responsibility. 5. They use the Flags sub-tab to see what they've flagged (and how QA resolved it). They use the Performance sub-tab to see their personal accuracy, flag rate, datasets finalized, and items labeled.

Reference screenshots: `screenshots/02_annotation_my_datasets.png` , `screenshots/03_annotation_flags_open.png` , `screenshots/04_annotation_flags_resolved.png` , `screenshots/05_annotation_performance.png` , `screenshot_annotation_view.png` , `Screenshot 2026-06-03 at 3.16.45 PM.png` , `Screenshot 2026-06-03 at 3.29.06 PM.png` , `Screenshot 2026-06-03 at 3.37.27 PM.png` , `Screenshot 2026-06-03 at 3.38.54 PM.png` .

Requirement	Details	User Story	Notes	Milestone
Annotator Identity Selector	Right-aligned dropdown (≈1/5 width) in the Annotation header. For Annotator-role users, the dropdown is locked to their own identity (or hidden). For Super Admins, the dropdown lists every member of the platform Annotator group and switching it switches the entire view into impersonation of that annotator.	As a Super Admin, I want to see what an annotator sees without sharing their login.	Must pull from the platform Annotator user group live (req 3.4). Impersonation actions are logged as Super Admin (req 3.5, 9.1).	M1
My Datasets Sub-Tab — Summary Cards	Four clickable cards across the top: Assigned (inbox icon, blue), Needs Attention (warning icon, red — counts <code>needs_revision</code> and datasets with open flags), In Progress (clock, amber), Complete (check, green — counts submitted/in QA/approved/finalized). Clicking a card filters the table.	As an annotator, I want to see my workload at a glance and jump straight to what needs me.		M1
My Datasets Sub-Tab — Filter Tabs	Full-width pill bar below the cards: All / Action Needed / In Progress / Submitted / Done. Click toggles the table filter.	As an annotator, I want to slice my queue without re-sorting.		M1
My Datasets Sub-Tab — Dataset Table	Columns and widths: Dataset (30%) — name plus the manager's revision note when status is <code>needs_revision</code>; Detection (20%); Progress (22%) — progress bar plus "X/Y labeled (Z%)"; Status (15%) — color-coded badge; Flags (13%) — open flag count or "—". Default sort puts <code>needs_revision</code> first, then datasets with open flags, then <code>in_progress</code>, then <code>assigned</code>, then everything else. Clicking a row enters the annotation view.	As an annotator, I want the most urgent work to surface without me hunting.		M1
Annotation View — Header	Back button (returns to My Datasets), large dataset name, detection name + progress counter ("Major Corrosion · 431/500 labeled (86%)"), and a read-only badge when the dataset is in a terminal status (<code>submitted</code> , <code>approved</code> , <code>finalized</code>).	As an annotator, I want to know what I'm working on and whether I can still edit it.		M1

Requirement	Details	User Story	Notes	Milestone
Annotation View — Image Panel (Left)	Toolbar: image counter (X/Y — image_id), Copy ID button, Zoom +/-, Reset, Prev/Next. Large viewport (~500 px tall, responsive). Zoom 1x–4x with drag-to-pan when zoomed. Keyboard nav: left/right arrows move images (disabled while a textarea is focused).	As an annotator, I want a fast, low-friction labeling experience.		M1
Annotation View — Secondary Review Card	If the item is not flagged: a "Flag for Secondary Review" button that opens a modal with a reason textarea and an "I'm certain I can't label this" checkbox. If already flagged: amber indicator, reason visible, and (in QA context only) a "Resolve Flag" button.	As an annotator, I want to surface uncertain items to QA without blocking the whole dataset.	The flag modal is the same component as the one in HiL Review (M2).	M1
Annotation View — Ground Truth Label Card	Horizontal button row: DETECTED / NOT_DETECTED / UNSET. Color-coded active state (purple for DETECTED, accent for NOT_DETECTED, neutral for UNSET). Clicking persists immediately.	As an annotator, I want one-click labeling.		M1
Annotation View — Attributes Card	Only renders when the detection has a <code>segment_taxonomy</code> defined. Multi-select toggle pills, one per taxonomy option (e.g. "major", "minor", "hairline" for corrosion).	As an annotator, I want to capture detection severity/type alongside the label.	Taxonomy is read from the detection record.	M1
Annotation View — Annotator Note Card	Textarea for free-form notes per image. Auto-saves on blur and on navigation away. Stored on <code>DatasetItem.image_description</code>.	As an annotator, I want to leave context that helps the next reviewer.		M1
Annotation View — Resolved Flag History	When an item that was previously flagged has been resolved by QA, show a card with the original reason, QA's resolution action, resolution note, and date.	As an annotator, I want to learn from how QA handled my flags.		M1

Requirement	Details	User Story	Notes	Milestone
Annotation View — QA Report Card	When the dataset's status is <code>approved</code> or <code>finalized</code> , render a card with the dataset's QA acceptance rate (correct samples / total reviewed) and total correction count from QA sampling.	As an annotator, I want to see how my work performed in QA.		<i>M1</i>
Read-Only Mode	When the dataset is <code>submitted</code> , <code>in_qa</code> , <code>approved</code> , or <code>finalized</code> , all editable controls (label buttons, segment toggles, notes textarea, flag button) are disabled and the read-only badge is shown in the header.	As an annotator, I want to be sure I'm not accidentally editing finalized work.		<i>M1</i>
Submit Dataset Action	A "Submit" CTA appears when 100% of items are labeled. On submit, the dataset's <code>qa_status</code> transitions from <code>in_progress</code> to <code>submitted</code> . Confirmation modal first ("This will hand the dataset to QA. You won't be able to edit it after submitting.").	As an annotator, I want a clear handoff moment.		<i>M1</i>
Flags Sub-Tab	Filter pills at top: Open (N) / Resolved (N). Below: list of the annotator's flags. Each row: image thumbnail (10×10 rounded), Image ID (monospace) + Dataset name (separated by middot), flag reason text, and either a resolution badge+note+date (resolved) or "Flagged [date]" + "Click to view" (open). Clicking opens the shared Image Preview Modal with a Flag Reason section highlighted in amber.	As an annotator, I want one place to track every flag I've raised and how it was resolved.	Match <code>screenshots/03_annotation_flags_open.png</code> and <code>04_annotation_flags_resolved.png</code> .	<i>M1</i>

Requirement	Details	User Story	Notes	Milestone
Performance Sub-Tab	Four summary cards with hover InfoTips: Accuracy ("Correct samples ÷ total reviewed samples"), Flag Rate ("Open flags ÷ total items assigned"), Datasets Finalized ("Datasets that completed full QA process"), Items Labeled ("Total images labeled"). Below, a Finalized Datasets table: Dataset, Items, Accuracy (color-coded — green ≥90%, amber ≥75%, red <75%), Corrections, Status. Empty state if no finalized datasets yet.	As an annotator, I want to see how I'm doing without asking my manager.	Match screenshots/05_annotation_performance.png	M1
Annotator-Role Tab Visibility	Users whose primary platform role is Annotator see only the Annotation tab in Detection Lab's navigation. All other tabs are hidden and their API routes return 403 to that role.	As an annotator, I shouldn't see surfaces I can't use.	Reinforces req 3.1, 3.3.	M1

Section 4: QA (Quality Assurance) Tab (M1)

Purpose. The QA tab is the manager-facing counterpart to Annotation. It is where flagged items are resolved, annotator work is sampled for accuracy, redundantly-labeled datasets are compared for discrepancies, and datasets move through the formal QA pipeline (Draft → Assigned → In Progress → Submitted → In QA → Needs Revision / Approved → Finalized). It also exposes per-annotator performance metrics and a full activity log.

Workflow. 1. The QA manager opens the QA tab to the Pipeline view and sees a Kanban-style board of every dataset by status. Cards show progress, open flag count, assignee, and last updated. 2. The manager uses Flags Queue to triage open flags: each flag opens the Image Preview Modal where they pick a resolution action (label_confirmed / label_corrected / attributes_corrected / image_removed / needs_discussion) and optionally leave a note. Bulk Resolve All / Dismiss All available for triage. 3. The manager uses QA Sampling to spot-check a finalized annotator's work: choose a strategy (Random / Stratified / Flagged / Discrepancy), set a sample size, generate the sample, and review item-by-item. Outcomes update annotator accuracy metrics. 4. For redundantly-labeled (linked) datasets, the manager uses the Discrepancy view to surface images where the two annotators disagreed and decide which label wins. 5. The manager uses Metrics & Logs to view per-annotator scorecards (accuracy, flag rate, label error rate, attribute error rate, discrepancy rate, correction rate) and a chronological activity log of every QA action. 6. Approved datasets land in Finalized, where they're exportable and become inputs to M2's runs.

Reference screenshots: screenshots/06_qa_dashboard.png , screenshots/06_qa_pipeline.png , screenshots/07_qa_flags_queue.png , screenshots/08_qa_metrics_logs.png , Screenshot 2026-06-03 at 3.55.50 PM.png , Screenshot 2026-06-03 at 3.58.16 PM.png , Screenshot 2026-06-03 at 4.21.55 PM.png , Screenshot 2026-06-03 at 4.22.06 PM.png .

Requirement	Details	User Story	Notes	Milestone
Sub-Tab Navigation	Icon-based sub-tab bar across the top: Pipeline / Flags Queue / QA Sampling / Discrepancies / Metrics & Logs / Finalized. Same active-state styling as elsewhere in the app.	As a QA manager, I want one tab with clear sub-sections, not six different tabs.		M1

Requirement	Details	User Story	Notes	Milestone
Pipeline View — Kanban Columns	Columns for each QA status: Draft, Assigned, In Progress, Submitted, In QA, Needs Revision, Approved, Finalized. Each column lists dataset cards. Card content: dataset name, detection name, assignee, open-flag badge, progress bar (<code>items_labeled / size</code>), last-updated timestamp. Card is clickable to open dataset details.	As a QA manager, I want to see where every dataset is in the pipeline.	Match <code>screenshots/06_qa_pipeline.png</code> .	M1
Pipeline View — Filters	Filter controls above the board: by Status (multi-select), Detection (single), Assignee (single from Annotator group).	As a QA manager, I want to narrow the board when we have a lot in flight.	Assignee dropdown sourced from Annotator group (req 3.4).	M1
Pipeline View — Manual Status Transitions	The manager can move a dataset to a different status from the card menu (e.g. "Send back to annotator" → <code>needs_revision</code> , "Approve" → <code>approved</code> , "Finalize" → <code>finalized</code>). Each transition writes an entry to the QA activity log.	As a QA manager, I want explicit control over the pipeline, not just automatic transitions.	Transitions must respect role permissions and log actor (req 9.1).	M1
Flags Queue — Open Flags Section	List of all flags with status=open. Per-page selector (10/25/50, right-aligned). Each row: thumbnail, Image ID + Dataset name, flag reason, date. Clicking opens the Image Preview Modal. Batch "Resolve All" / "Dismiss All" buttons at top with a resolution action dropdown and optional note.	As a QA manager, I want to triage the flag backlog quickly.	Match <code>screenshots/07_qa_flags_queue.png</code> .	M1
Flags Queue — Image Preview Modal (Flag Resolution)	Modal shows full image with zoom/pan, flag reason highlighted in amber, ground-truth label (editable), segment tags (editable), and a Resolution Action selector with options: <code>label_confirmed</code> , <code>label_corrected</code> , <code>attributes_corrected</code> , <code>image_removed</code> , <code>needs_discussion</code> . Optional resolution note. Resolve / Dismiss buttons at bottom.	As a QA manager, I want all context and action in one place.	Reuses the shared Image Preview Modal with flag-specific extensions.	M1

Requirement	Details	User Story	Notes	Milestone
Flags Queue — Resolved Flags Section	Collapsible section below Open Flags. Same row format but read-only and shows the resolution badge. Cannot be re-resolved.	As a QA manager, I want to look back at how a flag was handled without re-opening it.		M1
QA Sampling — Strategy & Sample Generation	Inputs: Dataset selector, Sampling Strategy (Random / Stratified / Flagged / Discrepancy), Sample Size. "Generate Sample" creates <code>QaSample</code> records.	As a QA manager, I want a reproducible way to spot-check annotator quality.	Strategy semantics: Random = uniform; Stratified = balanced by label; Flagged = only flagged items; Discrepancy = only items disagreeing with a linked sibling dataset.	M1
QA Sampling — Review Table	Table of sampled items: thumbnail, image ID, annotator label vs expected label (if discrepancy strategy), action buttons (Accept / Correct Label / Correct Attributes / Mark Both Corrected / Skip), notes field. Reviewing updates <code>QaSample.status</code> to "reviewed" and sets the outcome. Past samples for this dataset visible below.	As a QA manager, I want a fast review loop that produces auditable accuracy metrics.	Outcomes feed into Annotator Performance metrics.	M1
Discrepancies View	Linked-dataset pair selector. On selection, render a table of images where labels disagree between the two annotators' copies: Image ID, Annotator A label, Annotator B label, Tags A, Tags B, mismatch indicator. Clicking opens the Image Preview Modal with both labels shown. Resolution options: pick the winning label, or mark the image for removal. Trend chart of discrepancy rate over time.	As a QA manager, I want to surface and resolve label disagreements between annotators.	Only available when both datasets are linked via <code>linked_dataset_id</code> and share image IDs (req 4.4).	M1
Metrics & Logs — Sub-Toggle	Top of the view: Annotator Metrics / Activity Log toggle. Annotator Metrics is the default. When Metrics is active, a Detection filter dropdown is inline, right-aligned (~1/3 width).	As a QA manager, I want to switch between metrics and audit quickly.		M1

Requirement	Details	User Story	Notes	Milestone
Metrics & Logs — Annotator Metrics Table	Columns (each header has a hover InfoTip explaining the metric): Annotator, Datasets Assigned, Datasets Completed, Items Labeled, Accuracy ("Correct samples ÷ total reviewed samples"), Flag Rate ("Open flags ÷ total items assigned"), Label Error ("Label corrections ÷ total resolved flags"), Attr Error ("Attribute corrections ÷ total resolved flags"), Discrepancy ("Label disagreements ÷ total overlapping items"), Correction ("1 – accuracy rate"). Sortable columns.	As a QA manager, I want a single scorecard view per annotator.	Match <code>screenshots/08_qa_metrics_logs.png</code>. Annotator rows must reflect the live Annotator group membership.	M1
Metrics & Logs — Activity Log Table	Chronological log of QA actions: status transitions, sample reviewed, flag resolved, dataset approved, impersonation session, etc. Columns: Action, Actor, Dataset/Detection, Details, Timestamp. Filterable by action type.	As a Super Admin, I want a single auditable timeline of every QA action.	Source is the platform audit log (req 9.1). Impersonation sessions surface as "Super Admin X impersonated Annotator Y from ... to ...".	M1
Finalized View	Table of datasets with status <code>approved</code> or <code>finalized</code>. Columns: Dataset name, Detection, Size, QA Status, Accuracy (if sampled), Items Labeled, Approval date. Expandable rows show QA sampling results and flag resolution stats. Export button per row (CSV/JSON).	As a downstream consumer, I want one place to find datasets that are ready to use.	Once finalized, datasets show up as available inputs in the M2 Build & Run tab.	M1
No-Lock Concurrency on Annotation	Per the prior QA architecture decision: no locking on per-image annotation. Last write wins, but each label change writes an audit-log entry with actor and timestamp.	As a team, we tolerate concurrent labeling without blocking each other, and we can reconstruct the history if needed.	This matches the documented <code>project_qa_workflow_decisions.md</code> decisions.	M1
Un-Finalize Allowed	A finalized dataset can be moved back to <code>approved</code> or <code>needs_revision</code> by a Super Admin. The action is logged.	As a Super Admin, I want an escape hatch when a finalized dataset turns out to need rework.	Per the prior QA architecture decision.	M1

MILESTONE 2 — Prompt Building MVP

Milestone 2 layers the core prompt-building loop on top of M1's annotated datasets. After M2, a Prompt Engineer can author a detection, draft prompt versions, run them against finalized datasets, review predictions in HiL, and monitor runs centrally — but the automated improvement and final-eval features are still deferred to M3.

Tabs in scope: **Detection Setup**, **Build & Run Datasets**, **HiL Review**, **Detections & Logs**, **Admin**.

Section 5: Detection Setup Tab (M2)

Purpose. Detection Setup is where a detection (e.g. "Major Corrosion", "Missing Siding") is defined and its prompt versions are authored, tested with quick ad-hoc images, and approved as the current baseline. Every other prompt-related tab keys off the detection picked in this tab's workspace header.

Workflow. 1. The Prompt Engineer creates a new detection or selects an existing one. The detection record captures code, display name, category (`INCORRECT_CAPTURE` or `HAZARD_IDENTIFICATION`), description, label policy, decision rubric, optional segment taxonomy, and metric thresholds. 2. They author one or more prompt versions. Prompts are structured (detection identity / label policy / decision rubric / addendum / output schema / examples) and can be scaffolded from natural language via Prompt Assist (Gemini). 3. They use Quick Test to drop in a few ad-hoc images and see live inference results before committing to a full run. 4. They approve one prompt version as the current baseline. That version is what downstream tabs default to.

Reference screenshots: [Screenshot 2026-06-02 at 2.41.17 PM.png](#) , [Screenshot 2026-06-02 at 3.21.15 PM.png](#) , [Screenshot 2026-06-02 at 3.22.12 PM.png](#) , [Screenshot 2026-06-02 at 3.29.36 PM.png](#) , [Screenshot 2026-06-02 at 3.32.00 PM.png](#) , [Screenshot 2026-06-02 at 3.39.23 PM.png](#) .

Requirement	Details	User Story	Notes	Milestone
Detection CRUD	Form to create or edit a Detection: <code>detection_code</code> , <code>display_name</code> , <code>detection_category</code> , <code>description</code> , <code>label_policy</code> , <code>decision_rubric</code> (array of bullets), <code>segment_taxonomy</code> (optional array), <code>metric_thresholds</code> (precision/recall/f1 floor used for regression pass-fail). View mode renders read-only; Edit mode renders form.	As a Prompt Engineer, I want to capture detection metadata once and reference it everywhere.	Only Super Admin can delete a detection.	M2
Workspace Header — Detection Selector	Persistent header across all workflow tabs with a Detection dropdown. Changing detection re-scopes the entire workflow.	As a user, I want to stay on the same conceptual detection as I move through tabs.		M2
Prompt Version Table	Columns: <code>version_label</code> , <code>model</code> , <code>created_at</code> , <code>change_notes</code> , approval status badge. Actions per row: View, Edit, Duplicate, Approve as Baseline, Delete (Super Admin only).	As a Prompt Engineer, I want to see all prompt iterations for a detection in one place.	Approval is mutually exclusive — approving a new version unsets the prior.	M2
Create Prompt Version	"New Prompt Version" opens a modal with two options: Start Blank or Use Prompt Assist. Captures <code>version_label</code> , model selection (Gemini family), decoding parameters (temperature, <code>top_p</code> , <code>max_output_tokens</code>), and structured prompt sections.	As a Prompt Engineer, I want a fast path to a working first draft.	Model list pulled from Admin's allowed-models config.	M2

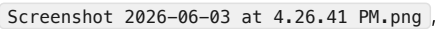
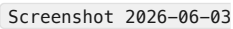
Requirement	Details	User Story	Notes	Milestone
Prompt Structure Editor	The user prompt is edited as discrete sections: detection_identity, label_policy, decision_rubric, user_prompt_addendum, output_schema, examples. Each section is its own labeled editor. Reassembled into a single prompt at runtime.	As a Prompt Engineer, I want surgical control of each part of the prompt without losing structure.	Default content for each section comes from the Admin category templates (see Section 9).	M2
Prompt Assist (Gemini)	A button in the create-prompt flow that takes the detection metadata + a short natural-language brief and calls Gemini to scaffold the prompt sections. User can accept, edit, or discard.	As a Prompt Engineer, I want a head start instead of staring at a blank editor.	Uses the Prompt Assist template defined in Admin.	M2
Quick Test	Drag-drop one or many ad-hoc images, run the current prompt against them, see results in-line (decision, confidence, evidence text, raw response). Results are not persisted.	As a Prompt Engineer, I want to sanity-check a prompt before queuing a full run.	Same model + parameters as the prompt version.	M2
Approve as Baseline	"Approve" button on a prompt version sets it as the detection's current baseline. The badge updates and downstream defaults point to it.	As a Prompt Engineer, I want a clear "this is the current best" marker.		M2
Regression Result Card	If a prompt version has been tested against the detection's golden set (from M3 Held-Out Eval flow), show pass/fail vs metric_thresholds plus the actual numbers.	As a Prompt Engineer, I want to see at a glance whether this version cleared the bar.	Populated by M3, but the card slot exists in M2.	M2

Section 6: Build & Run Datasets Tab (M2)

Purpose. Build & Run is where a Prompt Engineer either points an existing finalized dataset (from M1) at a prompt version and runs it, or — for one-off needs — builds a small ad-hoc dataset on the fly and runs it immediately. The output is a Run record with one prediction per image and a metrics summary if ground truth exists.

Workflow (Load mode). 1. Select an existing dataset for this detection. 2. Select a prompt version (defaults to the approved baseline). 3. Click Run. Watch the progress bar (live count / total with %). 4. On completion, see the metrics summary card and jump to HiL Review.

Workflow (Build mode). 1. Toggle to Build mode. Upload images (files / Excel / JSON). 2. Label each image (DETECTED / NOT_DETECTED / UNSET) and optionally apply segment tags. 3. Name the dataset, choose split type, save. (This creates the dataset just like Saved Datasets does, with the same image-ID uniqueness rules.) 4. Pick a prompt version and Run.

Reference screenshots:  ,  ,  .

Requirement	Details	User Story	Notes	Milestone
Mode Toggle	Load / Build toggle at the top of the tab.	As a Prompt Engineer, I want both "use an existing dataset" and "build something quick" paths in one place.		M2
Load Mode — Dataset Selector	Dropdown of datasets for the current detection plus an "Unassigned" group for cross-detection datasets.	As a user, I want to find the right dataset fast.		M2

Requirement	Details	User Story	Notes	Milestone
Load Mode — Prompt Selector	Dropdown of prompt versions for the current detection, with the approved baseline preselected.	As a user, I want the default to be the right answer most of the time.		M2
Run Execution	Clicking Run creates a Run record (status=running), kicks off Gemini inference per image, and streams progress. Cancel button available while running.	As a Prompt Engineer, I want to launch a run with one click and bail out if I see a problem.	Must handle 600-image runs end-to-end (req 8.2).	M2
Progress Display	Live X/Y count with percentage, current status badge, estimated time remaining (optional).	As a user, I want to know if I have time for coffee.		M2
Metrics Preview	On completion, show metrics_summary if ground truth is available: TP, FP, FN, TN, precision, recall, f1, accuracy, parse-fail rate. If no ground truth, hide metrics and show only the prediction count.	As a Prompt Engineer, I want immediate feedback on how the run went.		M2
Dataset Items Display	Below the metrics, a table of items: thumbnail, current ground-truth label, AI prediction badge, segment tags, "review flag" indicator (amber) if an annotator flagged that image. Clicking opens the Image Preview Modal with prediction details.	As a user, I want to spot-check results without leaving this tab.		M2
Build Mode — Upload Methods	Files (drag-drop), Excel/CSV (image_id, image_uri, ground_truth_label, segment_tags), JSON (structured array).	As a user, I want flexibility in how I get images in.	Image IDs must obey global uniqueness (req 4.4).	M2
Build Mode — Inline Labeling	Per-image label buttons (DETECTED / NOT_DETECTED / UNSET) and segment-tag toggles.	As a user, I want to label a small set without leaving the tab.		M2
Build Mode — Save Dataset	Captures name, split_type, optional auto-split (for MASTER). Persists like any Saved Datasets dataset.	As a user, I want my ad-hoc dataset to be reusable later.		M2
Quick Test Shortcut	Top-right "Quick Test" button mirrors the Detection Setup Quick Test — drop a handful of images, run current prompt, see results live, nothing persists.	As a user, I want a faster path than building a real dataset when I just need a smoke test.		M2
Cancel Run	While a run is running, the user can cancel; cancelled runs are persisted as <code>status=cancelled</code> with the partial predictions kept.	As a user, I want to abort a run that's clearly going sideways.		M2

Section 7: HiL Review Tab (M2)

Purpose. HiL Review is where a Prompt Engineer manually reviews model predictions from a Run, corrects labels, tags errors, flags ambiguous cases for secondary review, and watches metrics update live. Its output — the corrected labels and error tags — is the input to Prompt Feedback in M3.

Workflow. 1. Pick a completed Run for the current detection. 2. Filter predictions by error class (All / FP / FN / Parse Fail / Correct / Corrected / Flagged Open / Flagged Resolved). 3. Toggle between table view and image-by-image view. 4. For each item, confirm or correct the label, optionally assign an error tag, optionally leave a reviewer note, optionally flag for secondary review. 5. Watch the live metrics card update with the corrected confusion matrix.

Reference screenshots:  ,  .

Requirement	Details	User Story	Notes	Milestone
Run Selector	Sticky header dropdown of completed runs for this detection (most recent first).	As a Prompt Engineer, I want to switch between runs without losing context.		M2
Filter Tab Bar	Filters: All / FP (predicted DETECTED, GT NOT_DETECTED) / FN (predicted NOT_DETECTED, GT DETECTED) / Parse Fail (raw_response unparseable) / Correct / Corrected (manually adjusted) / Flagged Open / Flagged Resolved.	As a reviewer, I want to focus on the failure modes I care about.	Counts shown in each tab.	M2
View Toggle	Table view (paginated) and Image view (carousel with keyboard nav).	As a user, I want a power-user view (table) and a deep-focus view (image).		M2
Image View — Image Panel	Same image viewer as Annotation: zoom 1x–4x, drag-pan, arrow-key nav, X/Y counter, Copy ID, show/hide metadata.	As a reviewer, I want a consistent image-handling UX.	Reuse the Annotation image viewer component.	M2
Image View — Prediction Card	Right side: prediction badge (DETECTED / NOT_DETECTED / PARSE_FAIL), confidence score, evidence text, collapsible raw model response.	As a reviewer, I want full context on what the model said and why.		M2
Image View — Ground Truth Editor	Horizontal button row (DETECTED / NOT_DETECTED / UNSET) plus an "Update ground truth on the dataset" checkbox. If checked, the correction also writes back to the source DatasetItem (otherwise the correction only lives on the Prediction record for this run).	As a reviewer, I want to fix bad ground truth without doing it twice.		M2
Image View — Error Tag Selector	Dropdown with values: MISSED_DETECTION, FALSE_POSITIVE, INFERENCE_CALL_FAILED, AMBIGUOUS_IMAGE, LABEL_POLICY_GAP, PROMPT_INSTRUCTION_GAP, SCHEMA_VIOLATION.	As a Prompt Engineer, I want to classify why the model got it wrong so we can pattern-match later.	Error tags feed into Prompt Feedback's failure-cluster analysis (M3).	M2
Image View — Reviewer Note	Free-form textarea per item. Persists to the Prediction record.	As a reviewer, I want to leave notes for myself and the next person.		M2
Image View — Flag for Secondary Review	Same flag/resolve flow as Annotation. Flagging creates a ReviewFlag tied to this Prediction (and optionally the underlying DatasetItem). Resolution happens in QA's Flags Queue.	As a reviewer, I want to defer truly ambiguous calls to a second pair of eyes.	Reuses the Annotation flag modal.	M2

Requirement	Details	User Story	Notes	Milestone
Live Metrics Card	Updates in real time as corrections are submitted: confusion matrix, precision, recall, f1, accuracy.	As a reviewer, I want immediate feedback on whether my corrections move the needle.		<i>M2</i>
Batch Operations	From the table view: "Mark all in filter as correct", bulk error-tag assignment.	As a reviewer, I want to handle obvious cases in bulk.	Confirmation modal required for any bulk write.	<i>M2</i>

Section 8: Detections & Logs Tab (M2)

Purpose. A read-only operational view of every detection, every prompt version, every run, and every error — for monitoring, debugging, and exports. There is no editing here; this tab is for inspection.

Reference screenshots: project root [Screenshot 2026-06-02](#) series may include some of these views.

Requirement	Details	User Story	Notes	Milestone
Detection Accordion	Top-level list of all detections, expandable. Per detection: display_name, category badge, created_at, summary line ("X prompts, Y runs (Y completed, Z in progress)").	As a user, I want a directory view of the whole system.		<i>M2</i>
Prompt List per Detection	Inside an expanded detection: prompts table (version_label, model, created_at, regression result badge). Each row expandable to view the prompt structure.	As a Prompt Engineer, I want to see prompt history without context-switching to Detection Setup.		<i>M2</i>
Run List per Detection	Inside an expanded detection: runs table (run_id, prompt_version, dataset, model_used, status, progress, created_at). Color-coded status. Each row expandable to run details.	As a Prompt Engineer, I want to see all runs and their state.		<i>M2</i>
Run Details Panel	On expand: metrics summary, prompt snapshot (system + user prompt at time of run), decoding parameters, prompt_feedback_log (if M3 generated improvements), error list.	As a debugger, I want everything about a run in one place.		<i>M2</i>
Error List	Table of items that errored during the run: image_id, error type, error message, raw_response (first 200 chars). Filterable by error type.	As a debugger, I want to find broken images fast.		<i>M2</i>
Search & Filter	Search by detection_code or display_name. Filter runs by status.	As a user, I want to find a specific detection quickly.		<i>M2</i>
Exports	Export run as CSV (predictions + errors) and error report (error types, counts, sample messages).	As a user, I want offline analysis options.		<i>M2</i>
Delete Actions	Super Admin only: delete a run, delete a prompt version, delete a detection. Confirmation modal each time, listing what will cascade.	As a Super Admin, I want to clean up junk runs without involving engineering.	Reinforces role matrix (Section 1).	<i>M2</i>

Section 9: Admin Tab (M2)

Purpose. Admin is a Super-Admin-only tab for editing the global prompt templates and feedback-generation settings that the rest of the app reads from. There is no per-detection scope here — these are platform-wide defaults.

Requirement	Details	User Story	Notes	Milestone
Workbench Templates	Two editable templates: Prompt Assist Template (how new detections are scaffolded from a brief) and Prompt Feedback Template (how M3 generates improvement suggestions from HiL corrections).	As a Super Admin, I want to tune the meta-prompts our tool uses.		M2
Category Default Templates	Per detection category (<code>INCORRECT_CAPTURE</code> , <code>HAZARD_IDENTIFICATION</code>): default System Prompt and User Prompt that pre-populate new prompt versions for detections in that category.	As a Super Admin, I want category-appropriate starting points.		M2
Image Limits for Feedback	Integer inputs: max FP, max FN, max TP, max TN, max Parse Fail images to include in the context window when generating prompt-improvement suggestions in M3.	As a Super Admin, I want to control token costs and signal balance in the Feedback flow.	Used by M3 Prompt Feedback.	M2
Edit Mode + Save	Read-only by default with an Edit button. Edit mode has monospace editors and Save/Cancel. Last-saved timestamp shown.	As a Super Admin, I want explicit edit intent to avoid fat-fingering global defaults.	All saves audit-logged.	M2
Access Restriction	Tab itself is hidden to all non-Super-Admin roles; API returns 403.	As an org, I want only one role to be able to change platform-wide defaults.		M2

MILESTONE 3 — Prompt Enhancement & Evaluation MVP

Milestone 3 closes the loop: HiL corrections from M2 feed an automated suggestion generator (Prompt Feedback), prompt versions can be compared side-by-side (Prompt Compare), and a final regression gate (Held-Out Eval) blesses a version before it ships.

Tabs in scope: **Prompt Feedback**, **Prompt Compare**, **Held-Out Eval**.

Section 10: Prompt Feedback Tab (M3)

Purpose. Take a Run that's been reviewed in HiL and ask Gemini to propose surgical edits to the prompt sections that would have prevented the observed failures. The Prompt Engineer reviews, edits, and accepts suggestions; the tab then assembles a new prompt version and optionally smoke-tests it against the golden set.

Workflow. 1. Pick a completed, HiL-reviewed run. 2. Click "Generate Suggestions". The tool sends the prompt + failure samples (subject to the image limits set in Admin) to Gemini and gets back a structured list of suggested edits. 3. Each suggestion shows old text → new text, rationale, failure cluster, priority, risk, expected metric impact, expected parse-fail impact. The user checks the boxes they want, edits the new_text inline if needed, and creates a new prompt version. 4. Optional: run the new version on the golden set and see a pass/fail badge against metric thresholds.

Requirement	Details	User Story	Notes	Milestone
Run Selector	Sticky header dropdown of completed runs (most recent first), filtered to runs that have HiL corrections.	As a user, I want to feedback on runs I've actually reviewed.		M3
Generate Suggestions	Primary button. Calls Gemini with the prompt + sampled failure images (limits from Admin). Shows a loading state.	As a Prompt Engineer, I want a smart starting point for prompt edits.	Uses Admin's Prompt Feedback Template.	M3
Suggestions Table	Columns: select checkbox, section (which prompt slot), old_text → new_text comparison, rationale, failure_cluster, priority (1–5), risk (low/med/high), expected_metric_impact, expected_parse_fail_impact. Old/new_text inline-editable. Expand row for full text.	As a reviewer, I want enough context per suggestion to accept, edit, or reject confidently.		M3
Batch Accept/Reject	Buttons to accept all selected, reject all selected.	As a user, I want to triage long lists quickly.		M3
Create New Prompt Version	After accepting suggestions: input version_label, optional change_notes, "Create & Save". The new version's prompt sections are assembled by applying the accepted edits to the source prompt.	As a Prompt Engineer, I want one click to materialize my picks as a new version.	New version appears in Detection Setup's prompt table.	M3
Feedback Log Persistence	Accepted and rejected suggestions are persisted on the source Run (<code>prompt_feedback_log</code>) so future visits can see history and so audit trails are intact.	As a team, we want to know which suggestions we considered and rejected.		M3
Golden Set Regression Test	Optional: "Test on Golden Set" runs the new version on the detection's golden dataset, displays previous metrics vs new metrics, and renders a pass/fail badge against metric_thresholds.	As a Prompt Engineer, I want immediate validation that my edits help.	Result also propagates to Detection Setup's regression card.	M3
Pre-Filled Suggestions on Revisit	If a run already has a <code>prompt_feedback_log</code> , the tab loads the prior suggestions and selections instead of regenerating.	As a user, I want to pick up where I left off.	"Regenerate" button still available.	M3

Section 11: Prompt Compare Tab (M3)

Purpose. Side-by-side metrics comparison of multiple prompt versions on the same dataset, with a disagreement drill-down for images where the prompts diverged. Used to pick the best-performing version before Held-Out Eval.

Workflow. 1. Select up to 4 prompt versions via checkboxes. 2. Pick a dataset (the dataset selector only lists datasets where all selected prompts have completed runs). 3. View the metrics comparison table and a confusion matrix per prompt. 4. Drill into the disagreement images: each shows what each prompt predicted; click for the full preview modal with all prompts' predictions side-by-side.

Requirement	Details	User Story	Notes	Milestone
Prompt Multi-Select	Checkboxes listing prompt versions for the current detection. Hard cap at 4 selections; a 5th selection is disabled with a tooltip.	As a Prompt Engineer, I want to compare a small number of candidates head-to-head.		M3
Dataset Filter	Dropdown only shows datasets where *all* currently-selected prompts have completed runs. If the user selects a 4th prompt that has no overlap, the dropdown empties and a message explains why.	As a user, I want to be prevented from selecting an impossible comparison.		M3
Metrics Comparison Table	One row per prompt. Columns: TP, FP, FN, TN, Precision, Recall, F1, Accuracy. Cell shading highlights the best in each column.	As a reviewer, I want the winner to jump out visually.		M3
Confusion Matrices	One small matrix chart per prompt, rendered in a horizontal strip beneath the table.	As a reviewer, I want to see the shape of each prompt's errors, not just the headline numbers.		M3
Disagreement Analysis	List/grid of images where the selected prompts disagreed. Per row: thumbnail, image_id, ground truth (if known), and the decision from each prompt.	As a Prompt Engineer, I want to focus on where prompts actually diverge.		M3
Multi-Prompt Image Modal	Clicking a disagreement image opens the shared Image Preview Modal extended to show, in a side panel, the prediction (decision + confidence + evidence + raw response) from every selected prompt.	As a reviewer, I want to read each prompt's reasoning on the same image.		M3
Read-Only	This tab does not write anything (no corrections, no new prompts). Pure analysis.	As a user, I want to compare without fear of side effects.		M3

Section 12: Held-Out Eval Tab (M3)

Purpose. Run a prompt version against a held-out evaluation dataset (`split_type = HELD_OUT_EVAL`) as the final regression gate before deployment. Read-only results — no corrections allowed, no flagging, no re-runs against held-out data without explicit override.

Workflow. 1. Pick a prompt version. 2. Pick a held-out eval dataset (selector restricted to `HELD_OUT_EVAL` split type only). 3. Click "Run Evaluation". Watch progress. Cancel if needed. 4. On completion, view metrics, confusion matrix, segment-level metrics, parse-fail rate, and a pass/fail badge against metric_thresholds. 5. Optionally export the results as CSV/JSON.

Requirement	Details	User Story	Notes	Milestone
Prompt Selector	Dropdown of prompt versions for the current detection.	As a user, I want to evaluate any candidate, not just the baseline.		M3
Dataset Selector — Held-Out Only	Dropdown restricted to datasets with <code>split_type = HELD_OUT_EVAL</code> . Other split types are not selectable and an inline message explains why.	As an org, we want to enforce that held-out data stays held-out.		M3
Run Execution	Same Run pipeline as Build & Run, but with an <code>allow_eval_run=true</code> flag and write-protected predictions (no HiL corrections allowed against this Run).	As a Prompt Engineer, I want to use the same infra without accidentally muddying the held-out set.		M3

Requirement	Details	User Story	Notes	Milestone
Progress + Cancel	Live progress bar, optional ETA, Cancel button.	As a user, I want feedback during long runs and an out if I started the wrong one.	Must support 600-image runs (req 8.2).	<i>M3</i>
Results Display	Metrics summary (TP/FP/FN/TN, precision, recall, f1, accuracy), confusion matrix chart, segment-level metrics (if segment_taxonomy exists), parse-fail rate.	As a stakeholder, I want one screen that tells me "ship it or don't".		<i>M3</i>
Threshold Pass/Fail Badge	Compare results to detection's <code>metric_thresholds</code> ; render a pass or fail badge prominently.	As a reviewer, I want a clear go/no-go signal.	Result also surfaces on Detection Setup's prompt version row.	<i>M3</i>
Export Results	CSV / JSON download of the predictions and metrics.	As an analyst, I want offline records of the eval.		<i>M3</i>
History	Read-only table of prior eval runs on the selected dataset: prompt version, date, metrics, pass/fail.	As a team, we want a paper trail of every regression gate.		<i>M3</i>
No Corrections	The HiL Review tab does not list eval runs. Predictions on eval runs are immutable.	As an org, we want held-out data to remain a true test.		<i>M3</i>

Appendix A — Open Questions for Engineering

These are decisions intentionally left open for the implementation team. Each should be resolved during the M1 kickoff.

1. **Concurrency policy per entity.** Recommended split: optimistic locking on Detection / PromptVersion / Dataset metadata; last-write-wins on per-image labels and notes. Confirm. 2. **Parent-child dataset cascade.** When deleting a parent dataset that has linked children, refuse or cascade? Recommendation: refuse, force explicit unlink. 3. **Signed-URL strategy when sources are already Metabase signed URLs.** Confirm with Yong whether we re-sign or pass through. 4. **Rate-limit budget for the Gemini endpoint.** Must support a single 600-image sequential run; confirm per-user and per-org caps above that. 5. **Audit-log retention.** Confirm retention policy for QA activity log and impersonation records — recommend 1 year minimum. 6. **Annotator group authoritative source.** Confirm whether platform group membership is the only source, or whether Detection Lab also maintains a local allow-list for ad-hoc external annotators. 7. **Image-ID uniqueness scope.** Confirm uniqueness is per environment (dev / staging / prod isolated) vs truly global.